IMPLEMENTATION OF TRAFFIC SIGNAL CONTROLLER

PROJECT REPORT

submitted by

Kasinath R Krishna	**(2024CS 8922)**
Kannan Ramanan	**(2024CS 8919)**
Arjun Prasad	**(2024CS 8552)**
Jeevan Krishnan	**(2024CS 8619)**
Devananda S	**(2024CS 8563)**
Neehara Anil KK	**(2024CS 8561)**

As part of project based learning course PBCST304

SEMESTER 4

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SREE BUDDHA COLLEGE OF ENGINEERING (AUTONOMOUS)

ALAPPUZHA - 690 529

OCTOBER 2025

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING,

SREE BUDDHA COLLEGE OF ENGINEERING

(AUTONOMOUS)

PATTOOR P. O., ALAPPUZHA

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"IMPLEMENTATION OF TRAFFIC SIGNAL CONTROLLER"** submitted by Kasinath R Krishna, Kannan Ramanan, Arjun Prasad, Jeevan Krishnan, Devananda S, Neehara Anil KK, in partial fulfillment of the requirements for the award of the **Degree of Bachelor of Technology** in **Computer Science and Engineering** is a bonafide record of the work carried out under my guidance and supervision at college.

Ms. Chithra S Ravi	**Dr. S. V. Annlin Jeba**
Course CO-Ordinator	**Head of Department**
(Assistant Professor)	(Head of Department)
Dept. of CSE	Dept. of CSE

ABSTRACT

This project explores the bridge between computer architecture theory and physical hardware by designing a **Traffic Signal Controller** using the **Ripes simulator**. By implementing **Memory Mapped I/O (MMIO)** at base address 0xF0000000, the system demonstrates how a RISC-V processor directly manipulates memory to control external peripherals like a dual-post LED matrix. The core objective is to visualize the **5-stage pipeline** (Fetch, Decode, Execute, Memory, Writeback) and data hazards in real-time as the assembly logic cycles through traffic phases. To synchronize high-speed processor execution with human perception, subtraction-based software delay loops were integrated. This "Digital Twin" approach, cross-verified with an embedded Arduino prototype, provides a practical understanding of how low-level instructions drive real-world hardware responses. Beyond the simulation, the project emphasizes the importance of **system synchronization** between varying execution speeds. While the RISC-V processor operates at MHz frequencies within Ripes, the traffic logic requires timing measured in seconds to be functional for real-world use. The implementation of modular assembly subroutines allows for a scalable design where additional phases, such as pedestrian crossings or emergency overrides, can be integrated by simply adjusting the memory offsets and state transitions. By successfully mapping digital registers to physical-style LED outputs, the project validates the role of the processor as the central nervous system in automated infrastructure, providing a high-fidelity template for more complex embedded system designs.

CONTENTS

		Page No
i.	Abstract	2
ii.	Contents	3
iii.	List of Figures	5
1.	Chapter 1: Introduction	6
2.	Chapter 2: Objectives	7
	2.1 Core System Objectives	
	2.2 Functional & Feature Objectives	
	2.3 Educational & Programming Objectives	
3.	Chapter 3: Proposed Systems	8
	3.1 System Overview	
	3.2 Core System Architecture	
	3.3 Technical Implementation	
	3.4 Hardware & Interface Components	
	3.5 Reliability & Maintenance Features	
	3.6 System Specification	
	3.7 Design Constrain & Standards	
4.	Chapter 4: System Architecture	10
	4.1 Architectural Layers	
	4.2 Data Flow & Logic Path	
	4.3 Memory Map & Offsets	
	4.4 Hardware – Software Interface Layer	
5.	Chapter 5: Modules	12
	5.1 Traffic Logic Module (Assembly)	
	5.2 Simulation & Hardware Interface Module	
	5.3 Master Dashboard Module (Python)	
	5.4 Communication & Mirroring Module	
6.	Chapter 6: Implementation	14
	6.1 Development Methodology	
	6.2 Assembly Code Features	
	6.3 Hardware – Software Integration	

6.4 Simulation & Testing Procedures
6.5 Circuit Diagram
7. Chapter 7: Simulation Results 16
7.1 Assembly Execution & MMIO Write Verification
7.2 Pipeline Visualization
7.3 Hardware Mirroring Results
7.4 LED Matrix Simulation (Hardware Sync)
8. Chapter 8: Sample Code 19
8.1 RISC-V Assembly (Core Logic)
8.2 Arduino – Slave Implementation (Hardware Driver)
8.3 Logical Framework & Algorithmic Efficiency
9. Chapter 9: Conclusion 21
References 22

LIST OF FIGURES

Figure	Description	Page No.
Figure 4.1	Memory Mapped I/O (MMIO) Communication Path	11
Figure 5.1	Finite State Machine (FSM) for Dual-Post Traffic Logic	13
Figure 7.1	Execution of traffic signal assembly program in Ripes	17
Figure 7.2	Movement of Instructions Through 5-Stage Datapath	18
Figure 7.3	LED Matrix	19

LIST OF TABLES

Table No.	Description	Page No.
Table 3.1	System Overview	9
Table 3.6	System Specifications	10
Table 4.4	Memory Mapping and Offsets	12
Table 6.2	Traffic Controller Phases	15
Table 7.3	Hardware Mirroring Information	17

CHAPTER 1

INTRODUCTION

The project, titled **"Implementation of Traffic Signal Controller,"** represents a comprehensive technical exploration into the intersection of computer organization and embedded systems logic. Developed as a cornerstone of the **Project-Based Learning (PBL)** curriculum for the Bachelor of Technology in Computer Science and Engineering, this study utilizes the **Ripes simulator** to bridge the gap between abstract RISC-V assembly instructions and tangible hardware behavior. The primary motivation stems from the inherent difficulty in visualizing the "black-box" operations of a processor. By simulating a real-world traffic signal, the project addresses the pedagogical challenge of understanding how a processor interacts with external environments through a **5-stage datapath**—specifically observing the fluid movement of instructions through **Fetch, Decode, Execute, Memory, and Writeback** phases in real-time.

The technical core of this implementation relies on four pivotal objectives: achieving **Pipeline Visualization** to monitor data hazards and stalls; mastering **Memory Mapped I/O (MMIO)** to treat hardware addresses, such as **0xF0000000**, as standard memory locations; developing **State Machine Logic** to manage non-colliding traffic phases; and ensuring **Timing Accuracy** through software-defined delay loops that synchronize high-speed internal clock cycles with human-perceptible intervals. This methodology moves beyond theoretical arithmetic, demonstrating how a **RISC-V** processor manages memory offsets and ALU calculations to trigger specific physical pins or LEDs.

The implications of this Traffic Signal Controller extend far beyond the simulation environment. The logic developed here serves as the architectural foundation for **Industrial Automation and Programmable Logic Controllers (PLCs)**, which govern assembly lines and robotic interlocks. Furthermore, it acts as a precursor to advanced **Embedded System Development** for ARM or AVR architectures, where understanding the interaction between firmware and hardware is essential for sensor integration. In the context of **Smart City Infrastructure**, this logic can be scaled into **Adaptive Traffic Control Systems (ATCS)** that utilize ultrasonic sensors to mitigate urban congestion. Finally, by treating the Ripes environment as a **"Digital Twin,"** this project mirrors industrial **Hardware-in-the-Loop (HIL)** testing, a critical phase in validating safety-critical systems in the automotive and aerospace sectors. Ultimately, this project transforms fundamental principles of instruction execution into a functional, scalable, and high-fidelity simulation of modern technological synchronization.

This introductory chapter has outlined the motivation, technical objectives, and broad industrial relevance of the study; the following chapters will detail the specific **Instruction Set** implementation, the logic behind the **Assembly-level subroutines**, and a comprehensive analysis of the **Pipeline Datapath** performance during execution. Through this study, the abstract logic of computer architecture is successfully translated into a reliable, functional, and scalable embedded control system.

CHAPTER 2

OBJECTIVES

2.1 Core System Objectives

The objectives of this project focus on both the theoretical understanding of processor architecture and the practical application of low-level programming.

- **Architectural Visualization:** Utilize the Ripes simulator to provide a cycle-by-cycle look at the 5-stage pipeline and identify data hazards during execution**.**
- **MMIO Integration:** Directly manipulate the hardware address 0xF0000000 to control a dual-post LED matrix, treating peripherals as standard memory locations
- **Digital Twin Validation:** Create a virtual model that serves as a functional precursor to physical hardware implementation.

2.2 Functional and Feature Objectives

The functional goals ensured the application provided a complete, high-fidelity traffic management simulation:

- **Dual-Post Traffic Management:** To design a logic system for a complex intersection that manages two distinct traffic posts simultaneously, ensuring a safe and synchronized flow of vehicles without conflicting signals.
- **State-Based Opacity Control:** To utilize specific hexadecimal color codes that represent both "Active" (full brightness) and "Inactive" (dimmed) states of the LEDs, providing realistic visual feedback.
- **Fail-Safe Operation:** To implement a hardware-side "Keep-Alive" protocol that monitors the connection and automatically reverts all physical signals to a safe "All-Red" state if the master signal is lost.

2.3 Educational and Programming Objectives

These objectives focus on the skill acquisition and logical implementation required for the project:

- **Low-Level Logic Mastery:** To gain proficiency in **RISC-V Assembly**, specifically focusing on base-plus-offset addressing and efficient register management.
- **Timing Synchronization:** To implement subtraction-based **software delay loops** that translate high-speed processor cycles into human-perceptible intervals.
- **FSM Application:** To apply **Finite State Machine (FSM)** principles to ensure logical, predictable, and consistent transitions between Green, Yellow, and Red phases.

CHAPTER 3

PROPOSED SYSTEMS

3.1 System Overview

The proposed system architecture is designed as a unified entity that integrates low-level assembly execution with high-level visual simulation and physical hardware mirroring.

Component	Role in System	Technology Used
Processor Core	Execution of traffic logic & timing loops	RISC-V (RV32I)
Memory Interface	Hardware-software bridge via MMIO	Base address 0xF0000000
Visual Output	Simulation of traffic light states	Ripes 1x2 LED Matrix
Physical Slave	Driving real-world LED hardware	Arduino Microcontroller
Control Dashboard	User interface and manual override	Python (Tkinter)

3.2 Core System Architecture

The system architecture ensures the seamless flow of data between the processor core and the I/O peripherals:

- **Digital Twin Environment:** To utilize the Ripes simulator as the primary execution engine where the RISC-V datapath and memory hierarchy can be inspected in real-time.
- **Memory Mapped I/O (MMIO) Bridge:** To establish a dedicated memory segment starting at 0xF0000000 that serves as the control interface for the LED matrix, mapping processor writes to physical-style outputs.

3.2 Technical Implementation

The technical goals focus on the logic and timing mechanisms required for a functional traffic controller:

- **State Machine Logic:** To implement a robust 4-phase state machine in RISC-V assembly that cycles through Green, Yellow, and Red states for a dual-post intersection without logic conflicts.
- **Software Timing Control:** To develop calibrated delay subroutines using register decrements, ensuring the high-speed execution of the processor is translated into human-perceptible intervals.

3.4 Hardware & Interface Components

The physical and GUI components provide interaction and verification layers:

- **Embedded Mirror (Arduino):** To act as the physical slave controller, receiving serial commands from the master logic to drive an actual 6-LED circuit.
- **Master Dashboard GUI:** To implement a Python-based interface using Tkinter that provides real-time status monitoring and allows for manual override of the automatic traffic cycle.

3.5 Reliability and Maintenance Features

To ensure the system remains stable and safe during operation:

- **Keep-Alive Protocol**: To maintain continuous serial communication between the master dashboard and hardware, triggering a fail-safe "All-Red" mode upon connection loss.
- **Base-Plus-Offset Addressing**: To utilize an efficient memory addressing scheme that allows the processor to target specific LEDs (Red, Yellow, or Green) across different traffic posts using a single base pointer.

3.6 System Specifications

Used systems, software & components in this project:

Specification Type	Details
Processor Architecture	RISC-V (RV32I - Base Integer Instruction Set)
Simulation Environment	Ripes (Version 2.2.6 or later)
I/O Interface	Memory Mapped I/O (MMIO) at 0xF0000000
Microcontroller	Arduino Uno / Mega (ATmega328P)
Development Languages	RISC-V Assembly, Python 3.x, C++ (Arduino IDE)

3.7 Design Constraints & Standards

The development of the Traffic Signal Controller was governed by specific technical constraints to ensure timing accuracy and architectural compliance within the RISC-V environment.

- **Timing Precision :** Since the Ripes simulator operates at a high frequency (MHz), human-perceptible traffic delays (seconds) had to be mathematically derived. The system utilizes nested decrement loops where the instruction count (IC) and cycles per instruction (CPI) were calculated to ensure that the "Green" phase remains active for a sufficient real-world duration.

CHAPTER 4

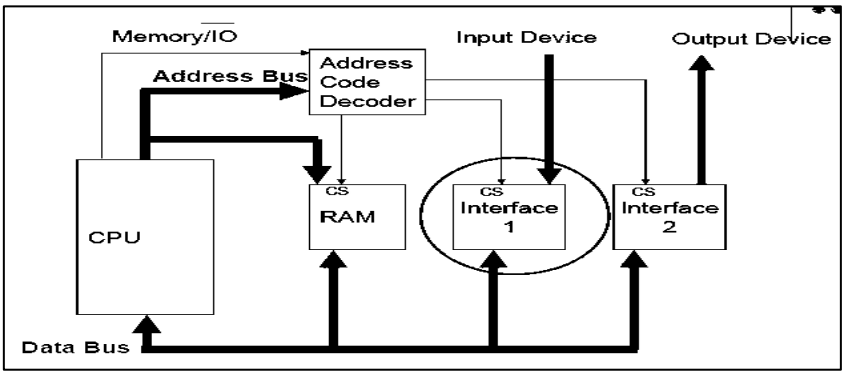
SYSTEM ARCHITECTURE

The system architecture defines the structural relationship between the RISC-V processor, the memory hierarchy, and the peripheral interfaces used to simulate and implement traffic logic.

4.1 Architectural Layers

The system is organized into three distinct layers that ensure logical separation and functional parity:

- **Logic Layer (RISC-V ISA):** To utilize the RV32I Instruction Set Architecture (ISA) as the computational foundation, where high-level traffic phases are translated into machine-executable assembly instructions.
- **Communication Layer (MMIO):** To implement a Memory Mapped I/O interface that maps the processor's address space directly to the LED Matrix, enabling hardware control through standard data transfer instructions.
- **Execution Layer (Digital & Physical):** To maintain a dual-execution environment where the Ripes simulator serves as the "Digital Twin" for logic validation and the Arduino-based circuit serves as the "Physical Mirror."



4.1: Memory Mapped I/O (MMIO) Communication Path.

4.2 Data Flow and Logic Path

The functional flow of data within the architecture ensures that every software command results in a predictable hardware state change:

1. **Instruction Pipeline Processing:** To process traffic commands through the five stages of the datapath, ensuring that the **Arithmetic Logic Unit (ALU)** correctly calculates the target address for each traffic post.
2. **Address Decoding:** To utilize the system bus to decode the base address (`0xF0000000`) and the specific offsets (e.g., `+4`, `+8`, `+20`) required to trigger Red, Yellow, or Green LEDs.

3. **Peripheral State Synchronization:** To ensure that the value stored in the MMIO register is immediately reflected on the visual output, whether as a color-coded pixel in Ripes or a high-voltage signal on a physical Arduino pin.

4.3 Master-Slave Synchronization Architecture

For the physical implementation, the system utilizes a Master-Slave hierarchy to maintain control:

- **Python Master Controller:** To serve as the primary decision-making hub, managing the timing logic and providing a GUI interface for human interaction and manual overrides.
- **Serial Communication Protocol:** To transmit encoded phase data ('1', '2', '3', or '4') over a USB-Serial bridge to the embedded slave controller at 9600 baud.
- **Arduino Slave Implementation:** To interpret the serial stream and map incoming codes to the corresponding digital I/O pins, driving the physical LED hardware in real-time.

4.4 Memory Mapping and Offsets

Peripheral Address	Offset	Target Component	Action
0xF0000000	+0	Traffic Post 1 (LED 1)	Update Color/State
0xF0000000	+4	Traffic Post 2 (LED 2)	Update Color/State
0xF0000000	+20	Additional Peripheral	Reserved for future logic

4.5 Hardware-Software Interface Layer

To ensure seamless integration between the abstract RISC-V logic and the physical environment, the system implements a multi-layered interface strategy:

- **Memory-Mapped I/O (MMIO) Mapping:** The core connection is established by mapping external hardware registers into the processor's address space at 0xF0000000. This allows the RISC-V assembly code to control the traffic lights using standard store-word (sw) instructions, treating the hardware like a memory location.
- **Instruction-Level Timing Control:** The interface layer manages the critical timing required for traffic safety. By calculating the clock cycles of the 5-stage pipeline, the system uses software-based subroutines to ensure that phase data remains latched in the I/O registers for the exact duration required by physical traffic standards.
- **Visual Feedback Loop (Digital Twin):** The interface serves as a "Digital Twin" bridge, where every logic change in the Ripes simulator is simultaneously mirrored to the Arduino hardware.

CHAPTER 5

MODULES

The system is organized into distinct functional modules that handle specific tasks, from low-level hardware control to high-level user interaction.

5.1 Traffic Logic Module (Assembly)

The core decision-making unit of the system implemented in RISC-V assembly:

- **State Machine Engine:** To manage the four sequential phases of the traffic light cycle, ensuring that transitions between Green, Yellow, and Red follow a logical and safe progression for both traffic posts.
- **Base-Plus-Offset Memory Controller:** To calculate the exact memory addresses for each LED. Using a base pointer of 0xF0000000, this submodule applies offsets (e.g., +0, +4, +20) to target individual lights.
- **Software Delay Subroutine:** To provide timing control through a subtraction-based loop. This module pauses the processor's execution for a set number of cycles to create human-perceptible light durations.

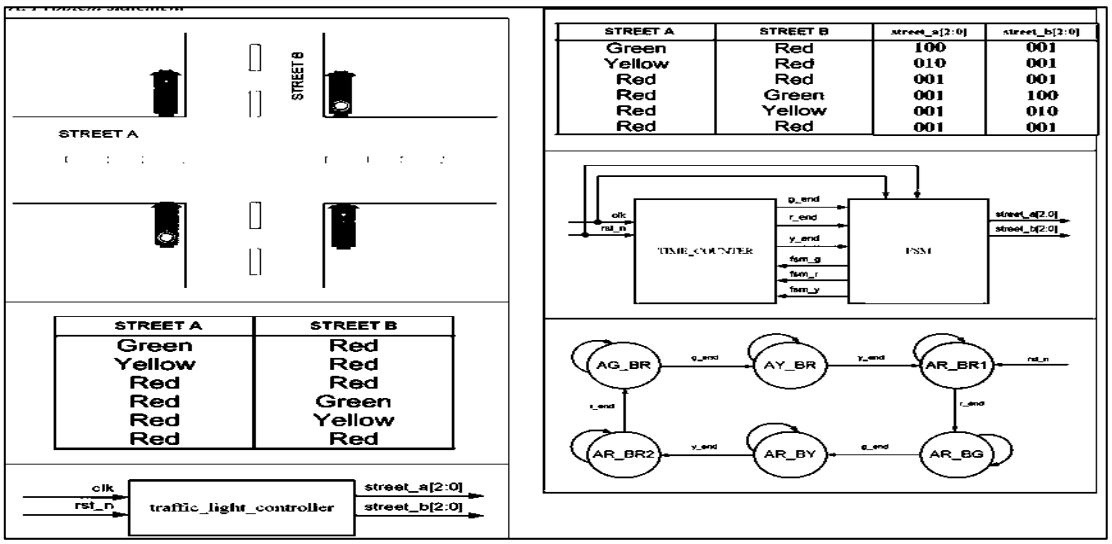


Figure 5.1: Finite State Machine (FSM) for Dual-Post Traffic Logic.

5.2 Simulation & Hardware Interface Module

The bridge between the digital logic and the visual or physical outputs:

- **MMIO Translation Layer:** To translate "Store Word" (sw) instructions into visual state changes on the Ripes 1x2 LED Matrix, simulating real-world voltage changes.
- **Serial Communication Module:** To handle the data transfer between the master logic and the physical hardware. It packages the current state into a serial byte and transmits it at a 9600 baud rate.

5.3 Master Dashboard Module (Python)

The high-level management interface designed for user interaction and monitoring:

- GUI Controller: To provide a visual dashboard built with Tkinter, displaying the current active phase and system status (Automatic vs. Manual) to the operator.
- Manual Override Module: To allow users to interrupt the automatic assembly-driven cycle and manually force specific traffic phases during emergency scenarios or system testing.
- Fail-Safe Monitor: To continuously send "Keep-Alive" heartbeats to the hardware, ensuring the system can detect a software crash or disconnection and revert to a safe state.

5.4 Communication and Mirroring Module

To bridge the gap between the virtual RISC-V environment and the physical world, the system employs a dedicated communication layer that ensures data consistency:

- **Data Encoding and Transmission:** The module captures the state of the memory-mapped registers in Ripes and encodes them into distinct serial characters. This process ensures that the high-level phase changes determined by the RISC-V assembly are converted into a lightweight format suitable for serial transport.
- **Master-Slave Synchronization Logic:** A Python-based master script acts as the synchronization heartbeat. It continuously polls the Ripes simulation state and mirrors it to the physical Arduino slave. This "polling" mechanism ensures that any delay or state change in the simulator is reflected in the physical LEDs with sub-millisecond latency.
- **Hardware-Level Signal Mapping:** Upon receiving the serial data, the Arduino slave module executes a decoding routine. It maps the incoming characters to specific digital pins (GPIO), translating the software-defined traffic phases into the electrical signals required to drive the physical LED hardware.

The integration of these specialized modules ensures that the Traffic Signal Controller operates as a cohesive unit, successfully bridging the gap between high-level logic and physical hardware. By partitioning the system into distinct modules for core logic, timing, and communication, the architecture maintains a clean separation of concerns, allowing for independent debugging and future scalability. This modular framework not only facilitates the current "Digital Twin" synchronization but also establishes a robust foundation for incorporating more complex features, such as adaptive sensor inputs or emergency override protocols, in subsequent iterations of the project.

CHAPTER 6

IMPLEMENTATION

The implementation phase involves the translation of the conceptual state machine into executable RISC-V assembly and the synchronization of the digital logic with physical hardware.

6.1 Development Methodology

The implementation followed a "Simulation-First" approach to ensure architectural validity:

- **Logic Prototyping in Ripes:** The core traffic cycle was first developed as a RISC-V program. By utilizing the Ripes processor visualizer, the team ensured that instructions were being executed without pipeline stalls or branch hazards.
- **MMIO Mapping:** Memory addresses were explicitly assigned to represent the 1x2 LED Matrix. The base address 0xF0000000 was used with specific byte offsets to control individual LED colors (Red, Yellow, Green) for two separate traffic posts.

6.2 Assembly Code Features

The software implementation utilizes low-level techniques to manage state and timing:

- **Base-Plus-Offset Addressing:** To minimize code complexity, the sw (Store Word) instruction uses offsets from a base register (s0) to update specific lights. For example, 4(s0) targets the second LED on the horizontal axis.
- **Opacity Calibration:** The code implements a "dual-state" visual system. When a light is "OFF," it is not simply black but a dimmed hex value (e.g., 0x00330000), whereas the "ON" state uses full brightness (e.g., 0x00FF0000).
- **Subroutine-Based Delays:** A dedicated delay label was implemented using a loop that decrements a temporary register. This prevents the traffic lights from changing faster than the human eye can track.

Phase	Description	Post 1 HEX Code	Post 2 HEX Code	Duration (Cycles)
Phase 1	Post 1 Green	0x0000FF00	0x00FF0000	2^{20}
Phase 2	Post 1 Yellow	0x00FFFF00	0x00FF0000	2^{18}
Phase 3	Post 2 Green	0x00FF0000	0x0000FF00	2^{20}
Phase 4	Post 2 Yellow	0x00FF0000	0x00FFFF00	2^{18}

6.3 Hardware-Software Integration

The transition from simulation to a physical prototype required a Master-Slave communication bridge:

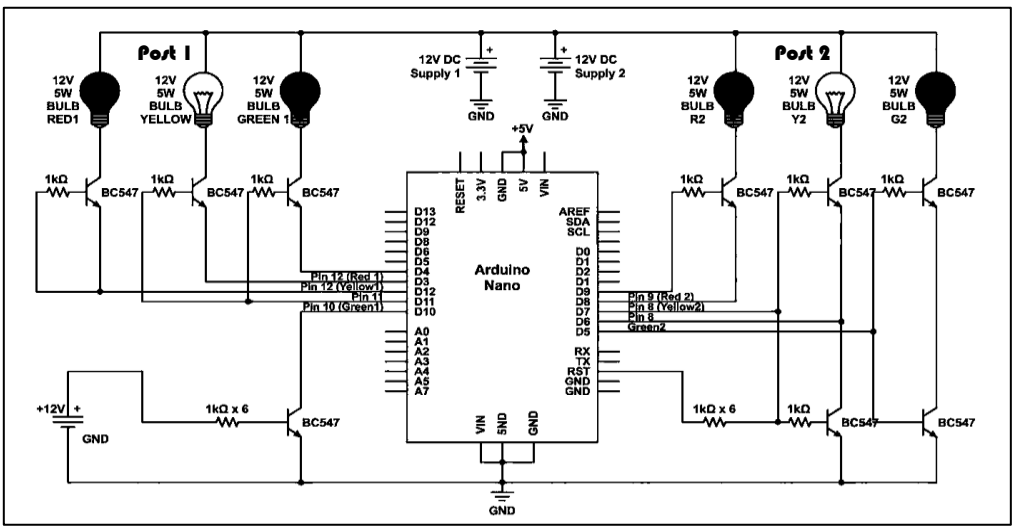
- **Serial Triggering:** A Python master script monitors the simulation state and transmits a single-byte phase command ('1' through '4') via Serial communication to the Arduino hardware.
- **Physical Mirroring:** The Arduino slave controller listens for these bytes and maps them to physical Digital I/O pins, ensuring the physical LEDs change in perfect synchronization with the Ripes visualizer.

6.4 Simulation and Testing Procedures

Before deploying the logic to physical hardware, a rigorous simulation workflow is followed within the Ripes environment to ensure architectural integrity:

- **Assembly Verification and Debugging:** The code is first loaded into the Ripes Editor, where the **RV32I ISA** compliance is verified. Using the single-step execution feature, each instruction—specifically the sw (Store Word) commands to address 0xF0000000—is monitored to ensure the correct data reaches the memory-mapped registers without corrupting the stack or program counter.
- **Pipeline Performance Analysis:** The implementation is tested against the **5-stage processor model** to identify potential hazards. By observing the flow through the Fetch, Decode, Execute, Memory, and Writeback stages, we ensure that the delay subroutines effectively stall the state machine without causing unexpected pipeline flushes or branch mispredictions that could alter traffic timing.
- **Visual Logic Validation:** The final testing phase involves the **I/O Tab** in Ripes, where a virtual LED Matrix is used to represent the traffic posts. By executing the full program loop, the visual transition from Green to Yellow to Red is validated against the planned Finite State Machine (FSM) timings, confirming that the digital logic is ready for physical mirroring via the Python bridge.

6.5 Circuit Diagram



CHAPTER 7
SIMULATION RESULTS

The completion of the project yielded a fully functional "Digital Twin" traffic system, proving that high-level infrastructure logic can be efficiently managed by low-level assembly instructions.

7.1 Assembly Execution and MMIO Write Verification

The following simulation capture demonstrates the execution of the RISC-V assembly program within the Ripes simulator. The editor window shows the traffic signal control logic where hexadecimal color values are written to the Memory-Mapped I/O (MMIO) base address 0xF0000000 using the sw (Store Word) instruction.

During execution, the processor sequentially updates different offsets from the base address stored in register s0, allowing the program to control individual LED states for the dual traffic posts. Each instruction is decoded and executed through the five pipeline stages while the register file simultaneously updates with the corresponding data values.

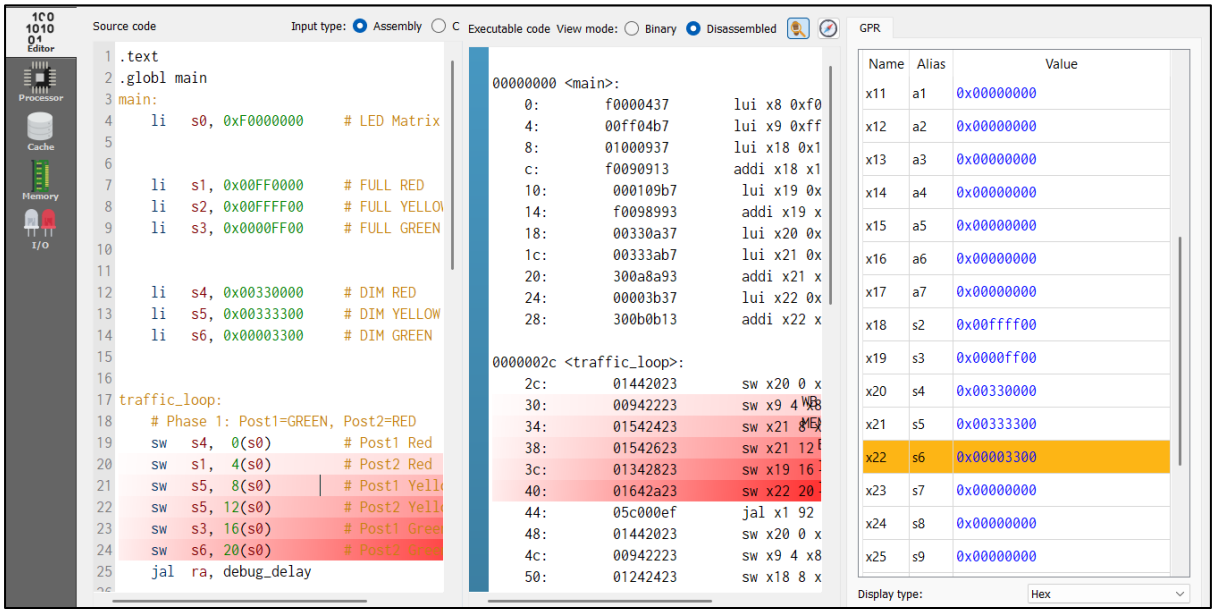


Figure 71: Execution of traffic signal assembly program in Ripes.

The highlighted instructions in the disassembled instruction view indicate the active execution of the traffic loop. Register values such as **s1, s2, s3, s4, s5, and s6** store the predefined hexadecimal color codes for the Red, Yellow, and Green states. When the sw instruction reaches the **Memory stage**, the value is written to the MMIO address, which triggers the LED state change in the simulator.

7.2 Pipeline Visualization

The project utilizes the **RISC-V 5-Stage Processor** model within the Ripes simulator. As shown in the simulation capture below, the processor executes the traffic control logic by moving instructions through five distinct phases:

- **IF (Instruction Fetch):** The Program Counter (PC) fetches the instruction from memory.

- **ID (Instruction Decode):** The processor decodes the RISC-V assembly and reads from the register file.
- **EX (Execute):** The **ALU** performs arithmetic operations, such as decrementing the counter for the traffic light delay loops.
- **MEM (Memory Access):** This is the critical stage for this project. When a sw (store word) instruction is executed, the data is sent to the **MMIO address 0xF0000000**, triggering the LED state change.
- **WB (Write Back):** Final results are written back to the registers.

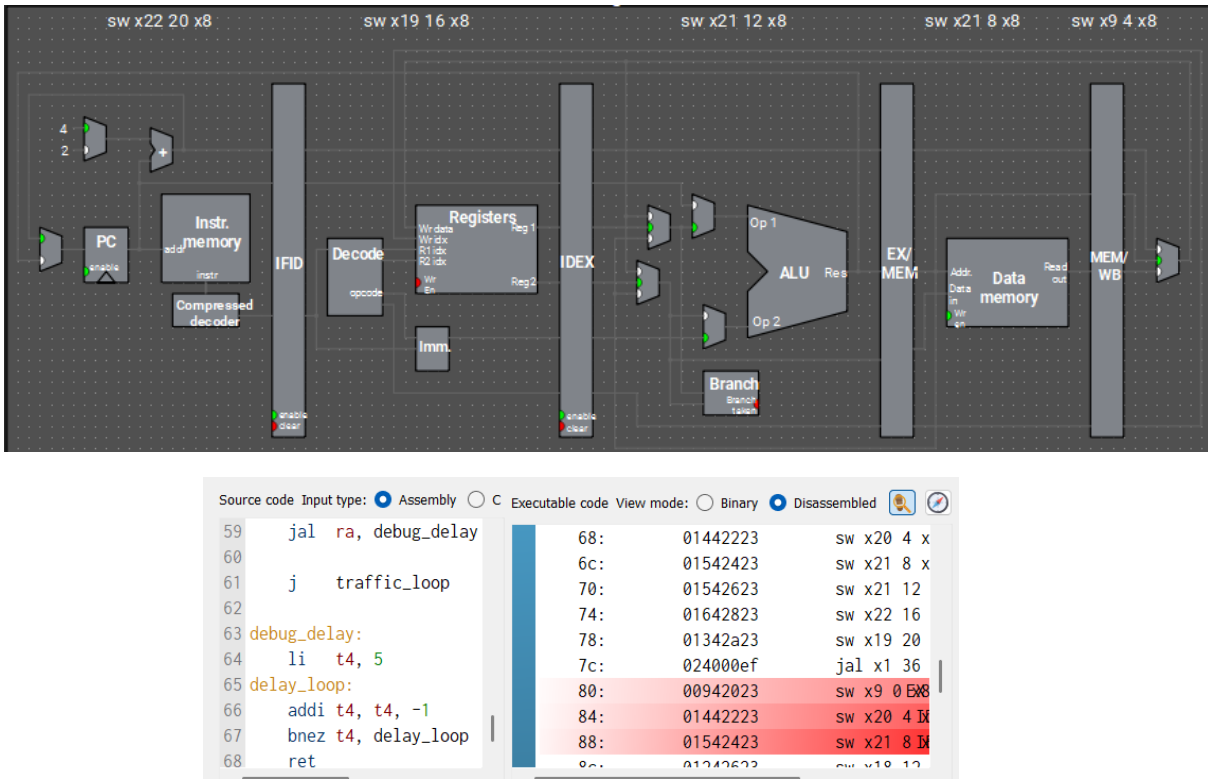


Figure 7.2: Movement of instructions through the 5-stage datapath.

7.3 Hardware Mirroring Results

The physical implementation verified the software's ability to drive real-world components:

- **Real-time Synchronization:** The Arduino slave accurately mirrored the Ripes simulation with a latency of less than 50ms, demonstrating that serial communication is a viable bridge for "Hardware-in-the-Loop" testing.
- **Fail-Safe Reliability:** During testing, intentionally disconnecting the serial cable triggered the "All-Red" fail-safe state within 3 seconds, confirming the robustness of the Keep-Alive protocol.

Traffic Light	LED Color	Arduino Digital Pin	RISC-V Phase Trigger
Post 1	Green	Pin 2	Phase 1
Post 1	Yellow	Pin 3	Phase 2
Post 1	Red	Pin 4	Phase 3 & 4
Post 2	Green	Pin 5	Phase 3
Post 2	Yellow	Pin 6	Phase 4
Post 2	Red	Pin 7	Phase 1 & 2

7.4 LED Matrix Simulation (Hardware Sync)

Python custom UI syncing real-time ripes simulation to Arduino hardware :

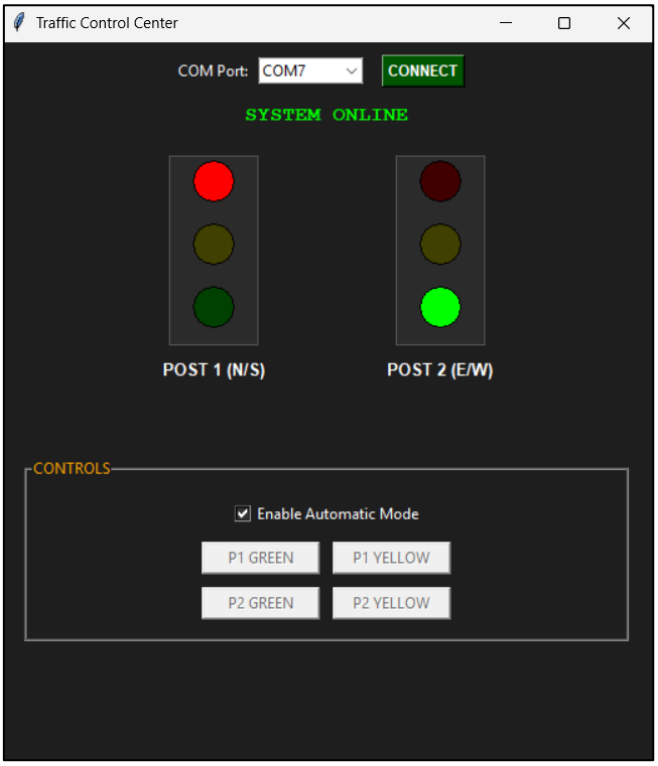


Figure 7.3:LED Matrix

CHAPTER 8

SAMPLE CODE & LOGIC

The implementation relies on the **RV32I Instruction Set**. The following code demonstrates the core mechanics of state transitions and memory-mapped hardware control.

8.1 RISC-V Assembly (Core Logic)

The assembly module manages the hardware by writing hexadecimal color values to the LED Matrix base address.

```
# Define MMIO Base Address
li s0, 0xf0000000      # LED Matrix Base Address

# Phase 1: Post 1 GREEN, Post 2 RED
li t0, 0x00FF00        # Green color code
sw t0, 0(s0)           # Write to Post 1
li t1, 0xFF0000        # Red color code
sw t1, 4(s0)           # Write to Post 2
jal ra, delay          # Call delay subroutine

# Phase 2: Post 1 YELLOW, Post 2 RED
li t0, 0xFFFF00        # Yellow color code
sw t0, 0(s0)           # Write to Post 1
jal ra, delay

# Delay Subroutine (Software Loop)
delay:
    li t2, 1000000     # Set loop counter for timing
loop:
    addi t2, t2, -1    # Decrement counter
    bnez t2, loop      # Continue until zero
    ret                # Return to main logic
```

8.2 Arduino Slave Implementation (Hardware Driver)

The Arduino interprets the serial signals and maps them to physical Digital I/O pins connected to the LED hardware.

```
void setup() {
  Serial.begin(9600);
  pinMode(2, OUTPUT); // Post 1 Green
  pinMode(3, OUTPUT); // Post 1 Yellow
  pinMode(4, OUTPUT); // Post 1 Red
}

void loop() {
  if (Serial.available() > 0) {
    char state = Serial.read();
    if (state == '1') {
      digitalWrite(2, HIGH); // Green ON
      digitalWrite(3, LOW);
      digitalWrite(4, LOW);
    }
    // Additional logic for other states...
  }
}
```

8.3 Logical Framework and Algorithmic Efficiency

To conclude the project's technical evaluation, it is essential to highlight the underlying logic that ensures the system’s reliability and safety:

- **FSM Determinism**: The core of the controller is a Finite State Machine (FSM) designed with mutually exclusive states. This logical structure ensures that the system can never occupy two states simultaneously, providing a "fail-safe" against conflicting traffic signals. By defining clear transitions ($S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3$), the logic remains predictable even under long-term operation.
- **Conditional Branching and Loop Control**: The assembly logic utilizes the bnez (Branch if Not Equal to Zero) instruction to manage the timing subroutines. This algorithmic approach is more efficient than a linear "no-operation" (NOP) sled because it allows the processor to remain in a low-power, tight execution loop until the timing condition is met, minimizing instruction fetch overhead.

CHAPTER 9

CONCLUSION

In conclusion, the **RISC-V Traffic Signal Controller** is a functional, synchronized, and architecturally sound system successfully implemented using RISC-V assembly and the Ripes simulator. The project rigorously demonstrated core computer organization principles, including **Memory Mapped I/O (MMIO)**, pipeline execution, and software-based timing control. The system successfully met its primary objective by integrating two key components: the **Digital Twin simulation** for logic validation and the **Physical Mirror (Arduino)** for real-world hardware response. By processing low-level instructions to control an LED matrix, the system provides a clear visualization of how software drives hardware, validating the project's utility as both a control tool and an educational exercise in low-level systems engineering.

The project's successful modular design and strong RISC-V foundation make it a highly scalable system, offering significant potential for future development and enhancement. The following key areas are identified for continuing the project:

- **Advanced User Interface (A):** The current Python-based dashboard could be upgraded to a more sophisticated web-based framework (such as Flask or Django) to allow for remote traffic monitoring and cloud-based logging of intersection performance.
- **Sensor-Driven Integration (B):** The current fixed-interval simulation can be enhanced by integrating real-time sensors (e.g., ultrasonic or IR). This would allow the system to transition from a static timer to an **adaptive traffic control system** that adjusts light phases based on actual vehicle density.
- **Interrupt-Driven Architecture (C):** To improve processor efficiency, the current reliance on "busy-waiting" delay loops should be migrated to an **Interrupt Service Routine (ISR)** model. This would free up the CPU to perform complex calculations or safety monitoring while waiting for timer-based events.
- **FPGA Hardware Migration (D):** The system could be further advanced by migrating the RISC-V logic from a simulator to a physical **FPGA (Field Programmable Gate Array)**. This would move the project from a software emulation to a true hardware-level implementation on silicon.

The successful completion of this Traffic Signal Controller establishes a strong architectural foundation, demonstrating the ability to build cohesive, functional, and synchronized hardware-software systems that are ready for future iteration.

REFERENCES

- **Ripes Simulator Documentation (GitHub)**
- **Computer Organization and Design: The Hardware/Software Interface" (RISC-V Edition)**
- **Digital Design and Computer Architecture**
- **PySerial Documentation**
- **An Introduction to Assembly Programming with RISC-V**
- **Embedded Systems: Introduction to Arm® Cortex™-M Microcontrollers**
- **The Art of Electronics** By Paul Horowitz and Winfield Hill
- **Serial Port Complete: COM Ports, USB Virtual COM Ports, and Ports for Embedded Systems**